



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 5

Team No.: 19

Project Title: Linux Service Process Supervisor

Team Members

Roll Number	Student Name	Signature
2520030380	Yamini	
2520030364	Lekhana	
2520030613	Anikha	

1. Abstract

Linux Service Process Supervisor is a Linux-based Operating Systems project that implements a lightweight, user-space supervisor (a miniature init/systemd-style daemon) which launches, monitors, controls, and automatically recovers long-running service processes. In conventional execution, a service that crashes, hangs, or is left as a zombie goes unnoticed until someone intervenes manually. This project addresses the problem by placing a supervisor process above the services and using Linux signals and process-control system calls to observe state changes in real time and restore failed services without human intervention.

The project focuses on key Operating Systems and system programming concepts such as **process creation and lifecycle, exec-based program loading, signal generation and handling, inter-process communication (pipes and FIFOs), process groups, sessions and job control, file I/O based logging, copy-on-write process creation, and thread-safe synchronization**. System calls such as `fork()`, `execvp()`, `waitpid()`, `kill()`, `sigaction()`, `setsid()`, `pipe()`, and `mkfifo()` are used to implement supervision of child services.

The proposed solution consists of a Linux-based application in C/C++ in which a supervisor process reads a service configuration, starts each service as a child, reaps terminated children through `SIGCHLD`, applies a restart policy with back-off, and accepts run-time commands (start, stop, pause, resume, restart, status, reload) through a named pipe and signals. The expected outcome is a functional demonstration of process supervision, providing practical understanding of how Linux manages process lifecycles, asynchronous events, and inter-process coordination.

2. Problem Statement

In a multitasking Operating System, long-running services such as web servers, workers, and loggers may terminate unexpectedly, hang, or leave defunct (zombie) child entries in the process table. Without a dedicated supervising mechanism, administrators must detect failures manually, and unmanaged services can lead to downtime, resource leaks, and uncoordinated execution.

This project addresses the problem of reliable service management using Linux process-control facilities. A supervisor process must create services, track their PIDs and states, detect exits and crashes, distinguish a normal exit from a signal-induced termination, and restart services according to a policy, all while remaining responsive to operator commands. It must also shut down cleanly, terminating each service and its descendants without leaving orphaned or zombie processes.

The proposed system uses Linux system calls, POSIX signal programming, pipes, FIFOs, process groups, and file I/O so that the supervisor can control and communicate with its services. This gives practical insight into the OS as a service layer, the process lifecycle, asynchronous notifications, and common process-management pitfalls.

The project aims to demonstrate that a compact user-space supervisor built on core Linux primitives can provide dependable, observable, and controllable service execution.

3. Objectives

1. To understand the OS as a service layer, including user space, kernel space, and the system-call interface used by a supervisor program (CO-1).
2. To create, execute, and terminate service processes using `fork()`, `execvp()`, `waitpid()`, and to study the process lifecycle and state transitions (CO-2).
3. To avoid common process-management pitfalls such as zombie processes, orphaned children, and restart storms (CO-2).
4. To implement signal-based control and notification using `kill()`, `sigaction()`, `SIGCHLD`, `SIGSTOP/SIGCONT`, `SIGTERM`, `SIGHUP`, and `SIGUSR1` (CO-3).
5. To use anonymous pipes for capturing service output and named pipes (FIFOs) for operator commands, and to manage process groups and sessions for job control (CO-3).
6. To record service events and output in log files using Linux file I/O system calls and file descriptors, and to observe per-service memory usage through `/proc` (CO-4, CO-5).
7. To use POSIX threads with mutexes and condition variables to protect the shared service table and coordinate monitoring activity (CO-6).

4. Proposed Methodology

The project will be designed and implemented as a Linux-based C/C++ application that supervises service processes using signals, pipes, and process-control system calls.

1. **Configuration:** The supervisor reads a service table (name, command, restart policy, maximum retries) and creates an in-memory record for each service.

2. **Process Creation:** For each service, the supervisor calls `fork()`; the child creates its own process group with `setsid()/setpgid()`, redirects output with `dup2()`, and loads the service program with `execvp()`.
3. **Process Identification:** The PID, PGID, start time, and state of every service are stored so that each service can be tracked and controlled individually.
4. **Signal Handling:** Handlers registered with `sigaction()` respond to `SIGCHLD` (child exit), `SIGTERM/SIGINT` (supervisor shutdown), `SIGHUP` (reload configuration), and `SIGUSR1` (dump status). Handlers only set flags, keeping them async-signal-safe.
5. **Monitoring and Recovery:** On `SIGCHLD`, the supervisor calls `waitpid(WNOHANG)`, inspects the exit status (`WIFEXITED`, `WIFSIGNALED`), and restarts the service using exponential back-off with a retry limit.
6. **Process Control and IPC:** Operator commands arrive through a FIFO created with `mkfifo()`. The supervisor uses `kill()` to send `SIGSTOP`, `SIGCONT`, or `SIGTERM` (escalating to `SIGKILL` after a timeout) to a service or to its whole process group.
7. **Logging and Resource Observation:** Service output is read through pipes and written to per-service log files using `open()`, `write()`, and `close()`; memory statistics are read from `/proc/<pid>/status`.
8. **Concurrency:** A monitoring thread (`pthread_create()`) periodically checks service health, and a mutex with a condition variable protects the shared service table.
9. **Execution Flow:** The overall flow will be: **Read Config** → **Fork Services** → **Register Signal Handlers** → **Monitor Children** → **Detect Exit/Crash** → **Restart or Report** → **Handle Operator Commands** → **Graceful Shutdown**.
10. **Testing:** The application will be run in a Linux terminal with test services that exit normally, crash, hang, and spawn children, to verify restart behaviour, signal control, absence of zombies, and clean termination. Tools such as `ps`, `strace`, and `valgrind` will be used for verification.

The methodology provides a practical demonstration of **process lifecycle management, signal-based control, IPC, file I/O, and thread synchronization** on Linux.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation – <code>fork()</code> , <code>execvp()</code> (CO-2, CO-4)	Creates a child for each service and loads the service program; copy-on-write makes forking efficient.
Process Identification – <code>getpid()</code> , <code>getppid()</code>	Retrieves PID and PPID for tracking and managing services.
Process Synchronization – <code>waitpid()</code> , <code>WNOHANG</code> (CO-2)	Reaps terminated services, prevents zombies, and reads exit status.
Signal Handling – <code>sigaction()</code> , <code>SIGCHLD</code> , <code>SIGHUP</code> , <code>SIGUSR1</code> (CO-3)	Detects service exit, reloads configuration, and reports status asynchronously.
Signal Sending – <code>kill()</code> (CO-3)	Sends control signals from the supervisor to a service or process group.
Process Control – <code>SIGSTOP</code> , <code>SIGCONT</code> , <code>SIGTERM</code> , <code>SIGKILL</code>	Pauses, resumes, and gracefully or forcibly terminates services.

Process Groups and Sessions – <code>setuid()</code> , <code>setpgid()</code> (CO-3)	Groups a service with its descendants for job-control style management.
IPC – <code>pipe()</code> , <code>mkfifo()</code> , <code>dup2()</code> (CO-3, CO-5)	Captures service output and receives operator commands.
File I/O – <code>open()</code> , <code>read()</code> , <code>write()</code> , <code>/proc</code> (CO-4, CO-5)	Writes logs and reads per-process memory statistics.
POSIX Threads – <code>pthread_mutex_t</code> , <code>pthread_cond_t</code> (CO-6)	Protects the shared service table and coordinates the monitoring thread.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030380	Yamini	Supervisor core: service configuration, process creation with <code>fork()/execvp()</code> , child reaping with <code>waitpid()</code> and <code>SIGCHLD</code> , restart policy with back-off (CO-1, CO-2).
2520030364	Lekhana	Signals and IPC: <code>sigaction()</code> handlers, <code>kill()</code> -based control (stop, continue, terminate), FIFO command interface, pipes, process groups and sessions (CO-3).
2520030613	Anikha	Logging and file I/O, <code>/proc</code> memory monitoring, pthread monitoring thread with mutex and condition variable, testing and documentation (CO-4, CO-5, CO-6).

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for developing, compiling, and executing the supervisor.
C / C++	Used to implement process creation, signal handling, IPC, logging, and thread synchronization.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
POSIX Libraries	Provides APIs for processes, signals, pipes, FIFOs, file I/O, and pthreads.
Linux Terminal / Shell	Used to compile, run, test, and monitor processes and signals.
ps, strace, valgrind, gdb	Used to inspect process states, trace system calls, and debug memory and logic errors.

8. Expected Outcome

The project is expected to produce a functional Linux-based service process supervisor that demonstrates how a parent process can start, monitor, control, and recover multiple service processes using signals and other core Linux mechanisms.

1. The supervisor will create service processes using `fork()` and `execvp()`, displaying the PID and PPID of each.
2. Terminated services will be reaped promptly through `SIGCHLD` and `waitpid()`, leaving no zombie processes.

3. Crashed or failed services will be restarted automatically according to the configured policy, with back-off and retry limits.
4. Operators will be able to start, stop, pause, resume, restart, and query services through the FIFO interface and signals.
5. Service output and supervisor events will be recorded in log files, and per-service memory usage will be reported.
6. The shared service table will be protected by mutexes, ensuring race-free updates between threads.
7. The supervisor will shut down gracefully, terminating all services and their process groups.
8. Overall, the project will provide practical understanding of Linux process management, signal-based IPC, job control, file I/O, and concurrency, while demonstrating how a reliable service supervisor is built from OS primitives.

9. GitHub Repository

GitHub Repository Link:

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: